ETL Designer Blueprint

A second proof of concept, a visual ETL tool like the first one but with configurable data sources. The user drags nodes onto a Nodify canvas, configures them, runs the pipeline with ETLBox, and generates a DLL that a different WPF application uses.

ETLBox 3.11.0 · Nodify 7.3 · WPF · drawn 2026-10-27

FEATURE 1 · DESIGNED

Left-hand palette of ETL nodes. Drag onto the canvas, connect ports to build the pipeline.

FEATURE 2 · DESIGNED

Source nodes for the standard ETLBox data sources, configured in the UI: address and port, protocol, database details, file paths, REST endpoints. The first slice is CSV.

FEATURE 3 · DESIGNED

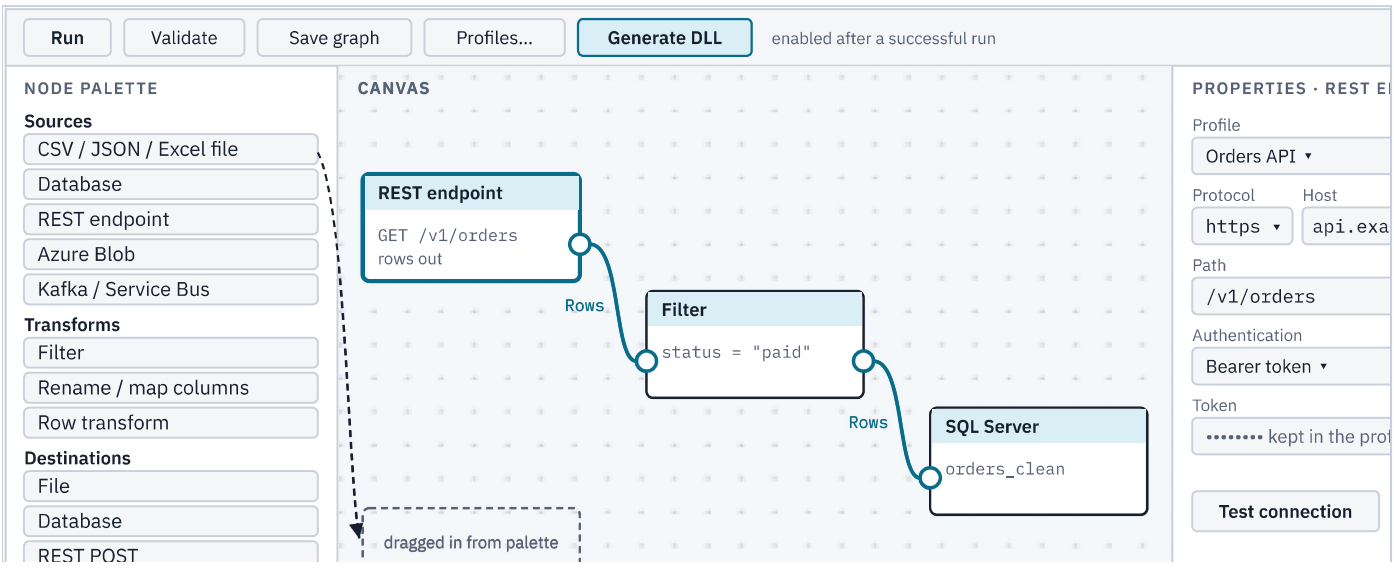
A Generate DLL button on the toolbar. The DLL is built with Roslyn, as in the first POC, and used by a different WPF application.

D read in ETLBox's public docs on 2026-10-27 P proven in our first POC

U not confirmed; check against the 3.11.0 packages

The screen

Four areas. The palette lists node types, the canvas holds the graph, the properties panel edits whichever node is selected, and the bottom tabs show validation, a data preview and the run log.



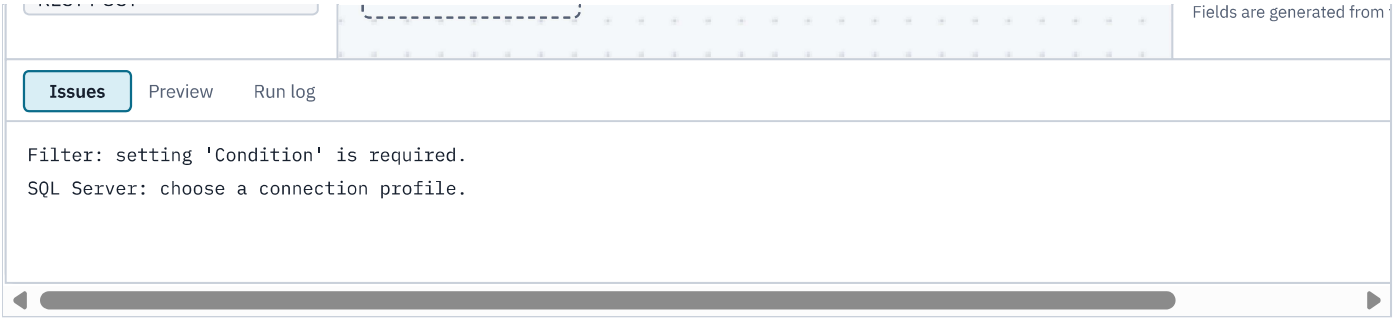


Figure 1. The REST node is selected, so the right panel shows its fields. The dashed outline is a node being dragged in from the palette. Wires carry rows between ports; the Issues tab lists what blocks a run. Generate DLL stays disabled until a run succeeds, as in the first POC.

First slice: CSV to a C# collection

Build one complete path first: a CSV file in, a collection out that a different WPF application can bind to. Databases and REST reuse the same shape afterwards.

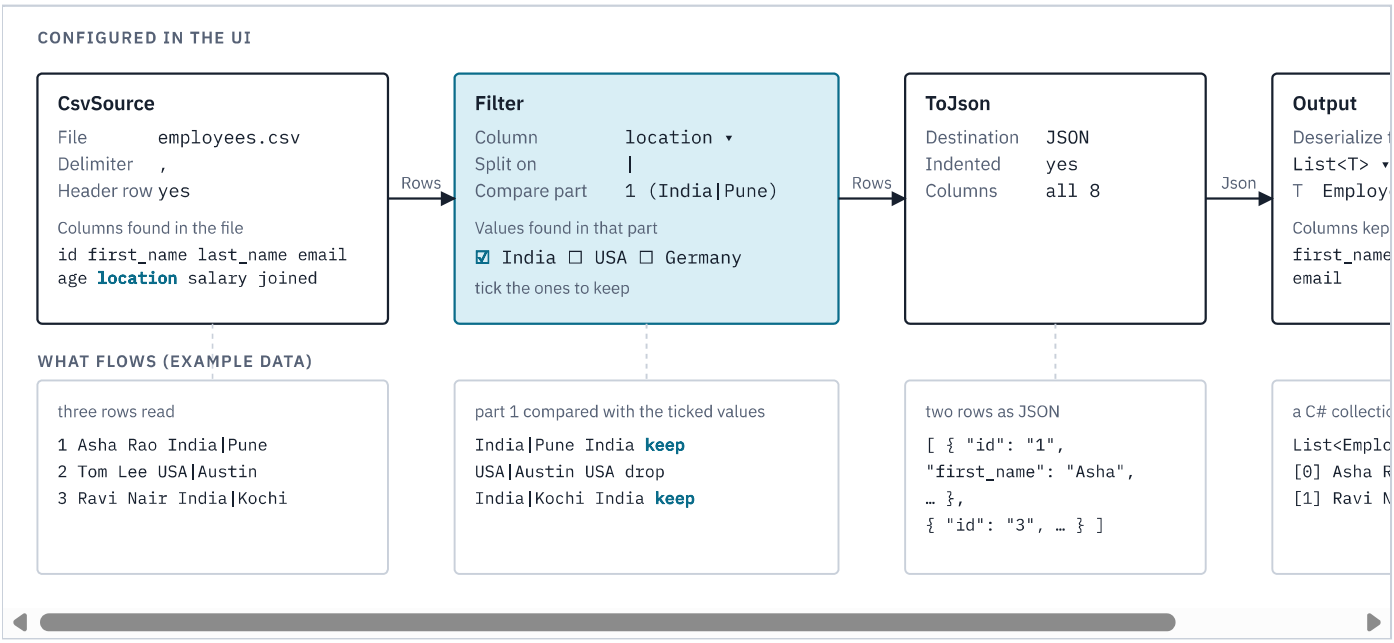


Figure 2. Top: what the user configures in each node. Bottom: what flows through it, with made-up example rows. The dashed lines only tie a node to its data; the arrows carry the rows.

1. **CsvSource.** Pick the file, the delimiter and whether there is a header row. The designer reads the header and lists the columns.
2. **Filter.** Pick a column, here `location`. If its values hold several parts, such as `India|Pune`, set the separator and which part to compare. The designer lists the values it finds in that part, and you tick the ones to keep.
3. **ToJson.** Writes the kept rows as indented JSON with all columns.
4. **Output.** Chooses what the JSON becomes in C#, and which columns it keeps.
5. **Run.** Each node shows its row count, and the Preview tab shows what it produced.
6. **Generate DLL.** The toolbar button builds the DLL once a run has succeeded.

Output type

A typed list is the default: the generator writes a class from the chosen columns, and a WPF DataGrid binds to it. A `DataTable` suits columns that are not known when the app is built. An `ObservableCollection` suits an app that edits rows. Plain JSON text stays available.

Column types

Start with every value as text, as in the first POC. Later each column can be marked int, decimal or date. A row that fails conversion is either reported and skipped, or stops the run. You choose which.

The values list

The designer reads the whole file to list every value in the chosen column. You will have an ETLBox licence, so the 5,000-row limit does not apply.

Using the DLL in a different WPF application

The app references the DLL and the ETLBox packages, and calls `Run`. The filter values saved in the designer are the defaults, and the call can override them.

```
var all     = GeneratedPipeline.Run(csvPath);
var india   = GeneratedPipeline.Run(csvPath, new PipelineArgs { Keep = new[] { "India" } });
EmployeeGrid.ItemsSource = india.Rows;   // IReadOnlyList<Employee>
```

A sketch of the call. The final names come from the generator.

The configurator rule: everything is a setting

Each node is a form with no hard-coded behaviour. Where the first POC fixed something in code (the case-insensitive match, the separator, an empty country accepting every row), the new tool shows it as a setting with a default. Settings that most people leave alone sit under an Advanced heading.

The Filter shows what this means. Figure 2 is the simplest case. The general form is a list of rules:

PART OF A RULE	CHOICES
Column	Any column the designer found in the file
Prepare the value	Nothing (a plain column), or split on a separator and take part N, trim, change case
Test	equals, is one of, contains, starts with, ends with, matches a pattern, greater than, less tha each can be negated

Value or values	Typed in, or ticked from the list of values found in the file
Case sensitive	On or off
Combine with the next rule	AND or OR

```
location   split on "|", part 1   is one of      India, USA        AND
age        (plain)                 greater than    30                AND
email      (plain)                 ends with       @example.com
```

Example rule list. The first rule is the country case from the first POC; the other two are plain columns.

Numbers and dates

Greater than and between need a number or a date, so those rules convert the text first. The rule also says what happens to a row that will not convert: skip it, or stop the run.

Source and output settings

CsvSource exposes delimiter, header row, and (once tested) quote character and encoding. Only the delimiter was proven in the first POC; encoding and header handling were open experiments there. Output exposes the class name, the columns kept and their types.

Settings the host can override

Any setting can be marked as a parameter. The saved value is the default, and the app that uses the DLL can replace it in its call to `Run`.

The cost of freedom

More settings mean more ways to build a pipeline that cannot run. The validator, the Issues tab and the Preview tab carry more weight: every setting needs a default and a clear message when its value is wrong.

C# expressions and parameters

A rule can be a typed C# expression instead of the form. The designer compiles it as you type. The generator compiles it into the DLL, so the app that uses the DLL needs no compiler.

```
row.age > p.MinAge && row.email.EndsWith(p.Domain)
```

Example Filter expression. It returns true to keep the row.

What an expression sees

`row` has one property per column, typed by the

In the designer

The expression is compiled while you type. Errors

column settings (text unless you set a type). `p` holds the declared parameters. A fixed set of namespaces is available (`System`, `System.Linq`, `System.Globalization`); more can be added under Advanced.

appear in the Issues tab with line and column, and Preview runs it on the sample rows. This is a new use of Roslyn inside the designer; the first POC used it only in the generator. I would use the same compile path as the generator, so a pipeline behaves the same in both places.

In the DLL

The expression text becomes a compiled method in the generated DLL. The host app does not need Roslyn at run time.

Safety

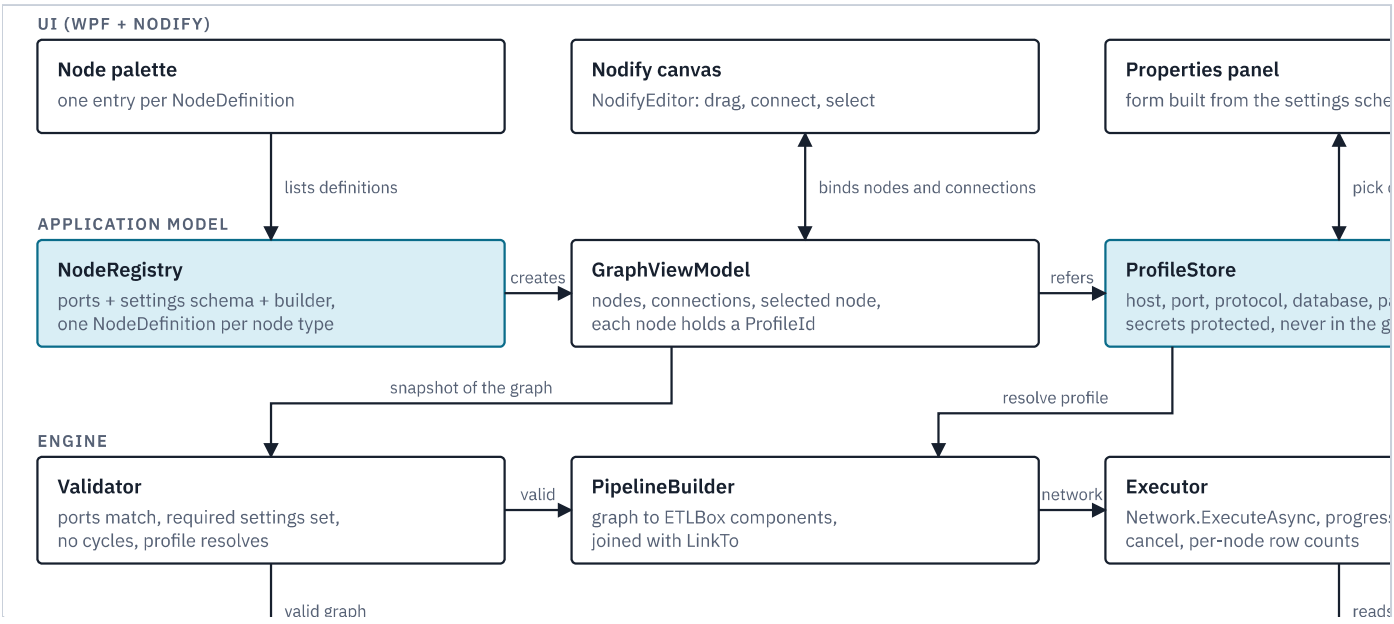
.NET has no sandbox for this. An expression runs with the permissions of the app that hosts it, and a saved pipeline file can contain code. Open only pipelines you trust, and show a warning when a file with expressions is opened.

Parameters

Declared in a Parameters panel: name, type, default. Any setting can take its value from a parameter, and expressions read them through `p`. Only declared parameters become fields of the generated `PipelineArgs`, so everything else is fixed when the DLL is generated.

How the parts fit

Two lookups drive the whole UI. The **NodeRegistry** says which nodes exist and what each one asks for. The **ProfileStore** says where each external system lives. A node saves only a profile ID, so addresses are entered once and passwords stay out of the graph file.



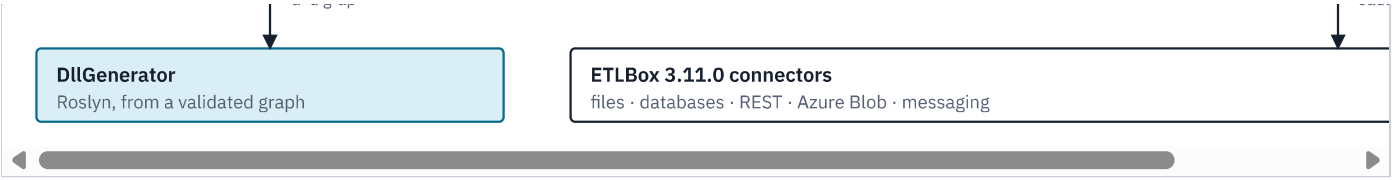


Figure 3. Adding a data source means adding one NodeDefinition (its fields and its builder). The palette, the properties form and the validator pick it up without new XAML. The DllGenerator accepts only a validated graph, so a DLL is never built from a broken pipeline.

Where host, port and protocol go

ETLBox does not have a host, port or protocol setting on its sources. It takes a file path, a URL, or a connection string. So the UI keeps typed fields (the ones you asked for) and a small composer per source type turns them into what ETLBox accepts. This is the main piece of code the POC has to get right.

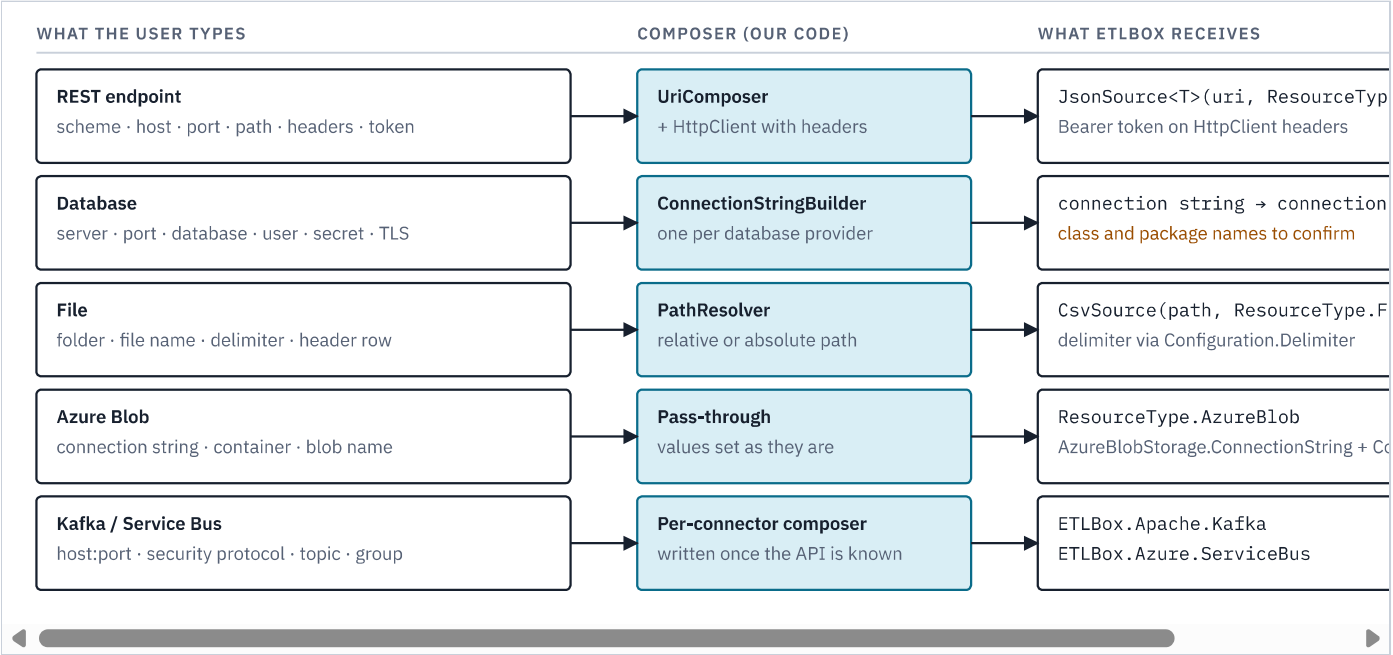


Figure 4. Left to right in every lane: typed fields become a composed value, which becomes the one thing ETLBox takes. Tags: D read in the docs, P proven in our first POC, U not confirmed.

"Protocol" has three meanings here

`http` or `https` for REST, a TLS option inside a database connection string, and the security protocol for Kafka. If you meant FTP, SFTP or S3: the ETLBox page I read does not list them, so they would need a custom source.

Docs are not pinned to 3.11.0

I read the current public docs. Before writing any builder, dump the public API of each connector package at 3.11.0 by reflection, as we did in the first POC. That settles every U in this page.

Generating the DLL

The first POC embedded its engine source in a Roslyn-compiled class library and ran the

same 52 tests through the designer and the DLL, with identical results. The new design keeps that and changes what gets embedded: the graph itself plus a general PipelineBuilder, instead of a fixed five-node chain.

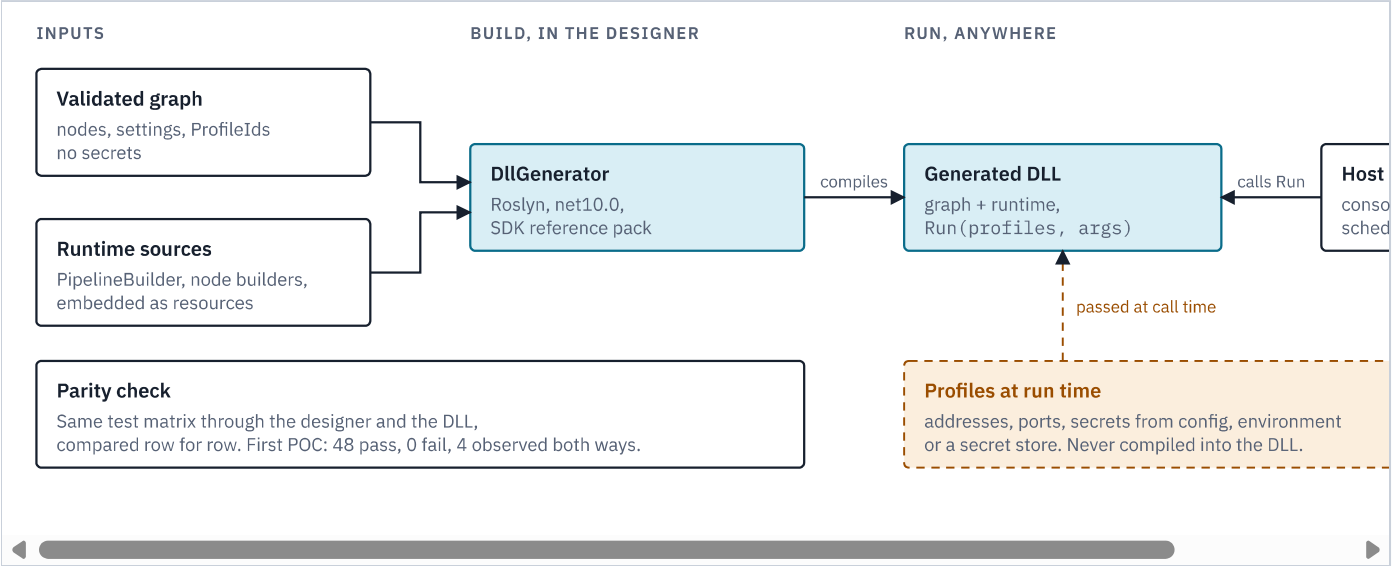


Figure 5. The generator accepts only a validated graph. Everything specific to an environment (addresses, tokens, passwords) arrives when the host calls Run.

There are two ways to produce the DLL. I recommend A.

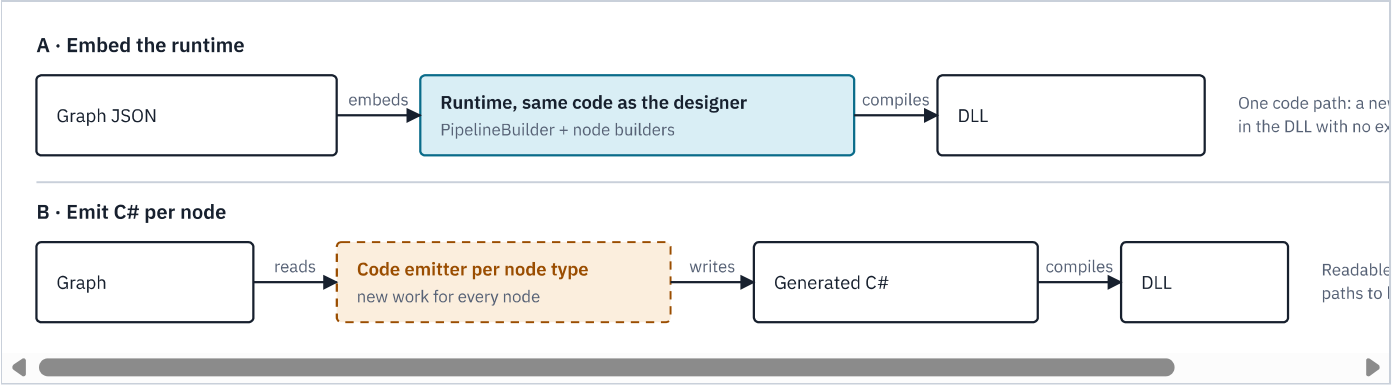


Figure 6. A reuses the designer's own builder, so there is one code path to test. B gives readable C# but needs an emitter for every node type, and each emitter can drift from what the designer does.

Packages and licence

The host needs ETLBox, ETLBox.Csv, ETLBox.Json and any connector package the graph uses. Above 5,000 rows read per connector it also needs a licence file.

Profiles

The DLL carries ProfileIds only. `Run` takes the profile values as an argument, or reads them from configuration or environment variables.

Generating versus running

Generating needs the .NET SDK reference pack, which the first POC found out the hard way. Running the DLL needs the .NET runtime and the

packages above.

Source catalog for the palette

What each source node asks the user for, and what I could confirm about the ETLBox side.

NODE	ETLBOX COMPONENT	USER FILLS IN
FILES		
CSV	`CsvSource`, package `ETLBox.Csv`	File path, delimiter,
JSON	`JsonSource`, package `ETLBox.Json`. The docs say it reads the first array it finds.	File path
Excel, Parquet, Text, XML	Packages `ETLBox.Excel`, `ETLBox.Parquet`, `ETLBox.Text`, and an XML package whose name the page spells `ETBox.Xml`. No class names on the page.	File path, plus shee Excel
WEB AND CLOUD		
REST endpoint (JSON)	`JsonSource` with `ResourceType.Http`; headers and auth set on its `HttpClient`. The docs describe GET reads and a POST destination, and no paging for reads.	Protocol, host, port, headers, bearer tok
Azure Blob	Any file source with `ResourceType.AzureBlob`	Connection string, c blob name
DATABASES		
SQL Server, Postgres, MySQL, MariaDB, Oracle, SQLite, Snowflake and others	The home page lists these as supported. It does not give class or package names.	Server, port, databa secret, TLS
MESSAGING		
Kafka, Azure Service Bus	Packages `ETLBox.Apache.Kafka`, `ETLBox.Azure.ServiceBus`. Docs name no classes or settings.	Bootstrap host:port protocol, topic, grou namespace connect and queue
LATER		
GraphQL, OData, SOAP, gRPC, ODBC, OLEDB	Listed on the home page, nothing more.	Not in the first relea
Transforms and destinations	From the first POC: `RowTransformation`, `JsonDestination`, `MemoryDestination`, `VoidDestination`. The docs say each	Expression or colun target path or URL

format has a destination.

Decisions to make before coding

Schema-driven forms

Each NodeDefinition lists its fields: type, required, secret, allowed values. The properties panel renders them. A new source is then one definition and one builder, with no XAML change.

Profiles, not node settings

Nodes store a ProfileId. Several nodes can share one profile, and the Profiles dialog is the only place addresses and secrets are edited.

Secrets

Protect stored passwords and tokens per Windows user, or store a reference to an environment variable. Mask them in the Issues tab and run log. Never write them into the saved graph.

Test connection

A button on each profile that opens the database, requests the URL or checks the file. A bad address then fails in the form, not halfway through a run.

Port kinds

Keep typed ports, as in the first POC, so a Rows output cannot be wired to a node that expects something else. The validator and the pending-wire colour use the same rule.

General graphs

The first POC ran one straight chain. A general ETL tool needs branches and joins, so the new PipelineBuilder must handle them. ETLBox has components for this; I have not checked them in 3.11.0.

Licence

The free edition read at most 5,000 rows per connector in the first POC. Every new connector node inherits that, so test data above that size needs a licence.

Build order

1. **API dump.** Reflect over every connector package at 3.11.0 and record classes and properties. Replace each U.
2. **The CSV slice.** Palette, canvas, NodeRegistry and a schema-driven properties panel, with CsvSource, Filter, ToJson and Output running end to end in the designer.
3. **DLL generation.** The toolbar button, the generator, and a small separate WPF app that references the DLL. Every source added afterwards is tested in the designer and in that app.